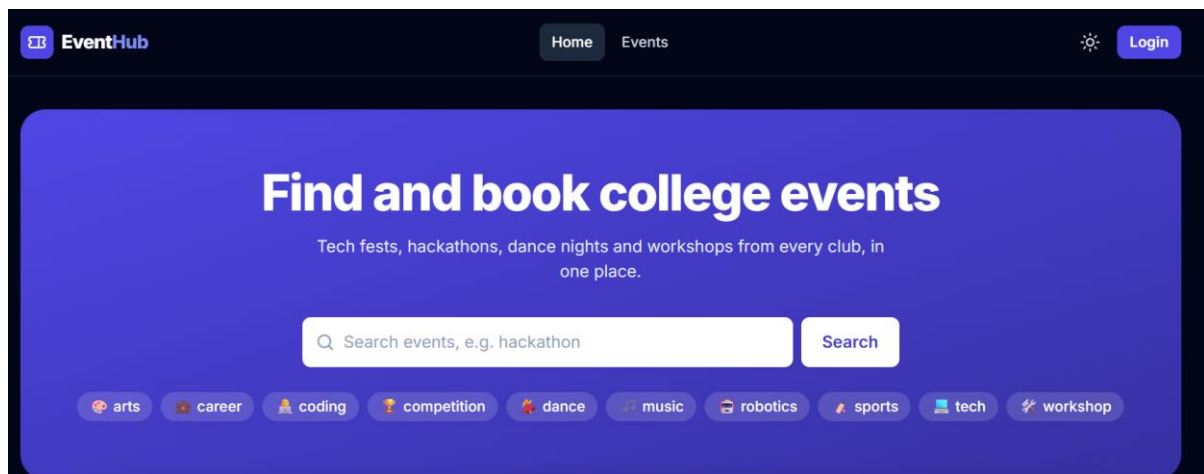


EventHub

College Event Management Platform

Explanation Notes and Viva Q&A

Simple scripts to explain EventHub to anyone, key concepts, numbers to remember and 62 viva questions with answers



Madhavan Suresh

Java Full Stack practice project · React + Spring Boot + MySQL

Version 1.0 · 29 September 2026 · github.com/madhavansuresh000-sys/eventhub

Contents

1. How to use these notes	2
2. Explanation scripts	3
2.1 The 1-minute explanation (elevator pitch)	3
2.2 The 5-minute explanation	3
2.3 For a non-technical person	4
2.4 For a teacher or examiner	4
2.5 For an interviewer	4
3. Key concepts in one line each	5
4. Hard problems and how I solved them	6
5. Numbers to remember	7
6. Viva questions with answers	8
6.1 About the project	8
6.2 Architecture	8
6.3 Backend and Spring	8
6.4 Concurrency and correctness	9
6.5 Security	10
6.6 Payments, waitlist, notifications, certificates	10
6.7 Frontend	11
6.8 Testing and quality	11
6.9 Database	12
6.10 Future and reflection	12

1. How to use these notes

These notes help you **explain EventHub to anyone** - a friend, a teacher, an interviewer or a non-technical person - in simple words. Read the scripts aloud a few times; then say them in your own words. Do not memorise every sentence; remember the **story** and the **numbers**.

If you have...	Use
1 minute	Section 2.1 (elevator pitch)
5 minutes	Section 2.2
A non-technical listener	Section 2.3
A teacher / examiner	Section 2.4, then chapter 6 (viva)
An interviewer	Section 2.5 and chapter 4 (hard problems)

2. Explanation scripts

2.1 The 1-minute explanation (elevator pitch)

"My project is **EventHub**, a website that runs college events from start to finish. Students find an event, book a seat, pay, and get a **QR ticket**. At the entrance a volunteer scans it - it works **only once**. After the event, students who really came get a **PDF certificate** that anyone can verify online, and they can rate the event. Clubs create events, and the college admin approves them.

I built it with **React** for the screens, **Spring Boot (Java)** for the server and **MySQL** for the data. The special part is that it stays correct when many people act at once: I proved with a test of **100 students clicking at the same second** that it never sells more seats than exist. It has **146 automated tests**, and GitHub runs them on every change."

2.2 The 5-minute explanation

1. **The problem (30 s)**. Colleges use WhatsApp and Google Forms for events. That leads to overbooking, shared ticket screenshots, late or fake certificates, and no numbers for organizers.
2. **The users (30 s)**. Visitors browse; students book; organizers create events for their club; volunteers scan tickets; the admin approves events.
3. **The journey (1 min)**. Ravi searches "hackathon", sees "110 of 120 booked", books one seat. The seat is **held for 10 minutes** while he pays. After payment he gets a QR ticket and an email. At the gate Priya scans it: green **LET IN**. If someone tries the same QR again: red **ALREADY USED at 8:41 pm by Priya**. After the event Ravi downloads a certificate with a random number like EH-2026-BEQR4Y79.
4. **How it is built (1 min)**. React app → Spring Boot REST API → MySQL. Inside the server: security filters, controllers, services with the rules, repositories for the database. Flyway keeps every database identical.
5. **The clever parts (1 min)**.

- No overselling: a **version number** on each event (optimistic locking) plus automatic retry. - One entry per ticket: **one SQL UPDATE** that only marks a ticket if it is not used yet. - Emails only **after** the database change is saved, so nobody gets an email about something that was undone. - Payments: seats held, Stripe (or a test page), a **signed** webhook that is processed only once, and automatic refunds for late payments.

6. **Security (30 s)**. BCrypt passwords, login token in an **httpOnly cookie**, CSRF protection, roles checked on every request, lock after 5 wrong passwords.
7. **Quality (30 s)**. 146 tests (Mockito, Testcontainers with real MySQL, React Testing Library), 91% coverage, CI on every push.

2.3 For a non-technical person

"You know how college events are announced on WhatsApp and people fill a Google Form? Sometimes more people fill it than there are seats, people share screenshots to get in, and certificates come weeks later.

EventHub is like **BookMyShow for college events**. You see all events in one place, you book your seat, and the website keeps your seat for 10 minutes while you pay - nobody else can take it. You get a QR code, like a movie ticket. At the door, a volunteer scans it with a phone: green means come in. If your friend tries the same QR code, it shows red, because each ticket works only once. After the event, you download a certificate, and any company can check on the website that it is real. The club that organised the event sees how many came and what people thought."

2.4 For a teacher or examiner

Stress the **engineering**: requirements, architecture, database design, testing.

- "It is a three-tier application: React SPA, Spring Boot REST API with layered architecture, MySQL with 17 tables managed by Flyway migrations."
- "Business rules are enforced on the server, inside transactions: for example the event status is a state machine - DRAFT, PENDING_APPROVAL, PUBLISHED - and illegal moves are refused."
- "Concurrency is handled with optimistic locking and verified with a 100-thread test; the gate uses a conditional UPDATE, verified with 20 parallel gates."
- "Security uses BCrypt, JWT in httpOnly cookies, CSRF tokens and method-level authorization with @PreAuthorize."
- "Quality: 133 backend tests with Testcontainers, 13 React tests, JaCoCo coverage 91% on services, GitHub Actions CI."
- "I documented it in a 105-page guide and a 21-slide presentation."

2.5 For an interviewer

Use the **STAR** pattern (Situation, Task, Action, Result) with one hard problem:

- **Situation:** "In my event platform, many students could try to book the last seats at the same moment."
- **Task:** "Guarantee we never sell more seats than exist, without making everyone wait."
- **Action:** "I used optimistic locking with a version column and a retry in a new transaction. I also found a MySQL deadlock and fixed it by always updating the event row before inserting the booking. I wrote a test where 100 threads book 10 seats at once, repeated three times."
- **Result:** "Exactly 10 bookings every time; without the version column the same test sold 96-99 seats. It runs in CI on every push."

Then offer a second story: the gate double-scan (conditional UPDATE + REPEATABLE READ snapshot), or the scanner crash your automated demo tour found.

3. Key concepts in one line each

Concept	One-line explanation	Where in EventHub
Full stack	Both the screens and the server, plus the database	React + Spring Boot + MySQL
REST API	URLs + HTTP methods that return JSON	50 endpoints under /api
Component	A reusable piece of screen	EventCard, ResultPanel
Redux	One shared store of app data	auth, cart, organizer, admin slices
Spring Boot	Java framework that wires everything up	EventhubApplication
Dependency injection	Spring gives each class what it needs	constructor + @RequiredArgsConstructor
JPA entity	A Java class for a table	Event, Booking
Repository	Interface to read/write a table	BookingRepository
Transaction	All changes succeed or none	@Transactional, TransactionTemplate
Flyway	Numbered SQL files that build the database	V1 ... V9
Optimistic locking	Save only if nobody changed it since you read it	@Version on Event and Booking
Conditional UPDATE	Check and change in one step	markCheckedIn
Idempotency	Doing it twice = doing it once	processed_payment_events
Outbox / after commit	Send emails only after the change is saved	EmailSender.afterCommit
JWT	Signed token saying who you are	EVENTHUB_TOKEN cookie, 8 hours
httpOnly cookie	Cookie JavaScript cannot read	the login cookie
CSRF	Attack from another website; blocked by a secret header	X-XSRF-TOKEN
BCrypt	Slow, salted password hashing	PasswordConfig
@PreAuthorize	Permission check on a method	@clubAccess.isOrganizer(#clubId)
Scheduled job	Code that runs on a timer	HoldExpiryJob, reminders at 18:00
Cache	Short-term memory of an answer	analytics, 60 seconds
Mock	Fake object for a test	Mockito in StripePaymentGatewayTest
Testcontainers	Real database in Docker for tests	MySQLTestDatabase
CI	Tests on every push	GitHub Actions
Code splitting	Load pages only when needed	React.lazy, 1213 → 329 kB

4. Hard problems and how I solved them

Problem	Simple explanation	Solution
Last-seat race	Two people read "1 left" and both book	@Version + retry; 100-thread test
Deadlock	Two transactions wait for each other forever	Always update the event first
Double scan	Two gates let the same ticket in	One conditional UPDATE; 20-gate test
Stale snapshot	MySQL showed the old "unused" value	Decide from the UPDATE result
Email for undone work	Email sent, then the change rolled back	Send after commit, retry on failure
Same webhook twice	Stripe resends a message	Unique message id table
Late payment	Money arrives after the hold expired	Take seats again or refund
Flaky JWT test	Failed 1 in 16 runs	Last base64 letter has 2 unused bits; change the first letter
Blank scanner page	Crash after a refactor	Found by demo tour; fix + page test + lint rule

5. Numbers to remember

Number	Meaning
10	phases (0-9)
29	pages / routes
50	REST endpoints
17	database tables, 9 migrations
146	automated tests (133 backend + 13 frontend)
91%	service test coverage (92% whole backend)
100 / 10	threads / seats in the concurrency test - exactly 10 bookings
20	gates scanning the same ticket - exactly 1 VALID
10 min	seat hold while paying
30 min	waitlist offer time
5 / 15 min	wrong passwords / lock time
8 h	login (JWT) lifetime
60 s	analytics cache
1213 → 329 kB	first download after code splitting
28	steps in the automated Edge demo tour
60+	Git commits

6. Viva questions with answers

Short, honest answers you can say in your own words. Grouped by topic; the most common questions come first in each group.

6.1 About the project

1. What is EventHub in one sentence? A college event platform where students find, book and pay for events, enter with a one-time QR ticket, and get a verifiable certificate, while clubs create events, the admin approves them and volunteers check tickets.

2. Why did you build it? Colleges run events on WhatsApp and Google Forms, which causes overbooking, shared ticket screenshots, late and fake-able certificates and no numbers for organizers. EventHub solves each of these with a rule in the software.

3. What is special compared with a simple CRUD project? It stays correct when many people act at once (no overselling, no double entry), it handles money (holds, payments, refunds, idempotent webhooks), and it has real-world features: waitlist offers, emails after commit, QR check-in, PDF certificates and cached analytics - all covered by automated tests.

4. Who are the users? Visitor, student, organizer (per club), volunteer (per club) and admin.

5. How long did it take and how did you organise it? Ten phases, from Java basics to launch, each with a checklist and "done when" checks; small commits, every push tested by GitHub Actions.

6.2 Architecture

6. Explain the architecture. Three tiers: a React single-page app in the browser, a Spring Boot REST API, and MySQL. Inside the backend: security filters, controllers, services, repositories.

7. Why separate controller, service and repository? Each has one job: the controller handles HTTP, the service holds the business rules and transactions, the repository talks to the database. Rules live in one place and each layer can be tested alone.

8. Why package by feature? Everything about one feature (for example booking) is in one folder, so it is easy to find, change and test.

9. How do the frontend and backend communicate? JSON over HTTP. The browser calls `/api/...`; the Vite dev server forwards it to port 8080. The login travels in an `httpOnly` cookie and the CSRF token in a header.

10. What is a REST API? A set of URLs with HTTP methods (GET to read, POST to create or act, PUT to replace) that return data, usually JSON, with status codes such as 200, 201, 400, 404 and 409.

6.3 Backend and Spring

11. What is Spring Boot? A Java framework that sets up a web server, database access, security and more automatically, so we only write the business logic.

12. What is dependency injection? Spring creates the objects (beans) and passes each class the beans it needs through its constructor, instead of the class creating them itself. It makes code loosely coupled and easy to test with mocks.

13. What does @Transactional do? It runs a method in a database transaction: all its changes are saved together, or none of them if an exception happens.

14. What is JPA / Hibernate? JPA is the standard for mapping Java classes to tables; Hibernate is the library that implements it and writes the SQL.

15. What is the N+1 problem and how did you avoid it? Loading a list (1 query) and then the details of each item separately (N queries). I used `@EntityGraph` to load the club with the events and `@BatchSize` to load all tags in one extra query.

16. How does search with filters work? With JPA Specifications: small WHERE-clause pieces (text, tag, club, dates) combined only when the filter is filled, plus server-side paging and sorting.

17. How do you handle errors? A global `@RestControllerAdvice` turns exceptions into Problem Details JSON with the right status: 400 validation, 404 not found, 409 business rule, 401/403 security, 429 login lock, 500 unexpected (details only in the log).

18. What is Flyway and why use it? It runs numbered SQL migration files in order and remembers which ran, so every database is identical. Hibernate only validates the schema.

19. What are scheduled jobs in your project? Every minute: expire unpaid holds, expire unanswered waitlist offers, retry failed emails. Every day at 18:00: reminders for tomorrow's events. They use `@Scheduled` and are switched off in tests.

6.4 Concurrency and correctness

20. How do you prevent overbooking? Optimistic locking: the event has a `@Version` column. An UPDATE only succeeds if the version is unchanged; the loser gets an exception and retries in a new transaction with fresh data, then sees fewer seats.

21. Why optimistic and not pessimistic locking? Optimistic locking does not hold database locks while the user thinks, scales better and conflicts are rare. Pessimistic `SELECT ... FOR UPDATE` would make everyone wait.

22. How did you prove it works? A test starts 100 threads for 10 seats at the same moment, three times: exactly 10 bookings each time. Without `@Version` the same test sold 96-99.

23. What deadlock did you meet? Two booking transactions locked the event and booking rows in different orders and waited for each other. The fix was to always update the event row first (`saveAndFlush`) before inserting the booking.

24. How does the gate stop a ticket being used twice? One conditional UPDATE: set `checked_in_at` only WHERE `checked_in_at` IS NULL. If it changed 1 row, let them in; if 0 rows, it was already used. A test with 20 gates proves exactly one VALID.

25. What is REPEATABLE READ and how did it affect you? MySQL's default isolation: a transaction keeps seeing the snapshot from its first read. The losing gate re-read the ticket and still saw "unused", so I decide from the UPDATE's row count instead of re-reading.

26. What is idempotency and where do you use it? Doing something twice gives the same result as once. Stripe may send the same webhook twice; each message id is inserted into a table with a primary key, so the second one is ignored. PUT for feedback is also idempotent.

6.5 Security

27. How is the password stored? As a BCrypt hash with a random salt; never in plain text.

28. What is a JWT and what is inside yours? A signed token with the user id, email, name, roles and expiry (8 hours), signed with HMAC-SHA256 using a secret key.

29. Why store the JWT in an httpOnly cookie? JavaScript cannot read it, so a malicious script cannot steal the login. localStorage would be readable.

30. Then how do you stop CSRF? SameSite=Lax cookies plus a CSRF token: the server sets a readable XSRF-TOKEN cookie, Axios copies it into a header, and other websites cannot read our cookie to forge the header.

31. Authentication vs authorization? Authentication proves who you are (password, then JWT). Authorization decides what you may do (roles and club rules, checked on every request).

32. Why are club roles not in the JWT? So that removing an organizer works immediately; a token would keep the old role until it expires.

33. How do you stop password guessing? Five wrong passwords for the same email from the same address lock that pair for 15 minutes (429 with Retry-After).

34. Can a student open admin pages by typing the URL? The page shows "no permission", and more importantly every admin API call returns 403, because the server checks the role.

35. How do you keep secrets out of GitHub? They live in `.env`, which is git-ignored; `.env.example` has placeholders; I scanned the code and the whole git history.

6.6 Payments, waitlist, notifications, certificates

36. How does payment work? Seats are held for 10 minutes, the server creates a Stripe Checkout session (or the built-in test page in development), the student pays on Stripe's page, and Stripe's signed webhook confirms the booking. A verify-on-return call is the backup.

37. Why the PaymentGateway interface? The booking code does not depend on Stripe. The same code works with Stripe or the fake test gateway, and a new provider (Razorpay) only needs a new class.

38. What if the payment arrives after the hold expired? The code tries to take the seats again; if they are gone, the payment is refunded automatically.

39. Explain the waitlist. When an event is sold out, students join a queue. Freed seats are offered to the first students that fit and kept for them 30 minutes; if not accepted, they go to the next.

40. Why send emails after commit? If the transaction rolled back after sending, the student would get an email about something that did not happen. So the notification is saved in the transaction and the email is sent after the commit, with retries.

41. How are certificates protected against faking? Only checked-in students of ended events get one; the number is random; anyone can check a number on the public verify page.

42. How are QR codes made? The server draws a PNG with ZXing. The QR contains only the random ticket code, nothing private.

6.7 Frontend

43. Why React? Component-based UI, a huge ecosystem, and fast updates of only the changed parts.

44. What is Redux used for? Data many pages need at once: the logged-in user, held seats, notifications, the organizer's events, the admin's queue. Async thunks call the API with pending/fulfilled/rejected states.

45. How do you protect frontend routes? A `RequireAuth` wrapper: spinner while checking, login page (remembering `?next=`) when not logged in, a friendly message when not allowed. The backend still checks every call.

46. How does form validation work? `useForm` validates with small rule functions, shows errors after leaving a field or on submit, focuses the first error and can show errors returned by the server.

47. What is code splitting? Loading some pages only when they are opened (React.lazy). It cut the first download from 1,213 kB to 329 kB.

48. How did you make it accessible and mobile-friendly? Labels on all inputs, `role="alert"` for scan results, a table view for charts, a 375 px phone check of every page, and a contrast check (4.5:1) in light and dark mode.

6.8 Testing and quality

49. What kinds of tests do you have? Unit tests with Mockito, integration tests with MockMvc and a real MySQL from Testcontainers, concurrency tests, React component tests with Vitest and Testing Library, and a scripted end-to-end tour.

50. What is a mock? A fake object that returns what the test says, so one class can be tested alone.

51. Why Testcontainers instead of H2? H2 behaves differently from MySQL (locking, SQL details, isolation). Testcontainers runs the same MySQL 8.4 as development.

52. What is your coverage? 91% of the service code, 92% of the whole backend, measured with JaCoCo.

53. What is CI? GitHub Actions runs all backend and frontend checks on every push, so a broken change is noticed at once.

54. Tell me about a bug you found. The gate scanner went blank after the first scan. My automated Edge tour found it: while refactoring for tests I had moved a colour table to another file but one list still used it. I fixed the import, added a page test that fails without the fix, and switched on the lint rule for undefined variables.

55. Tell me about a flaky test. A JWT tamper test failed about 1 in 16 runs. It changed the last base64 letter of the signature, but that letter has two unused padding bits, so sometimes the bytes did not change and the token was still valid. Changing the first letter fixed it.

6.9 Database

56. How many tables and which are the most important?

17. The core are users, clubs, events, bookings and payments; then waitlist_entries, notifications, certificates, feedback and audit_log.

57. Why VARCHAR + CHECK instead of ENUM? Easier to add a value later, readable, and maps naturally to Java enums stored as strings.

58. What constraints protect the data? Primary and foreign keys, UNIQUE (email, ticket code, one certificate and one rating per booking, one processed webhook id), CHECK (statuses, quantity 1-10, rating 1-5) and NOT NULL.

59. Why keep available_seats instead of counting bookings? One number is fast to read and, with the version column, easy to keep correct; counting bookings on every request would be slow and race-prone.

6.10 Future and reflection

60. What would you improve? Deploy it, add password reset, a real volunteer management page, UPI payments, an offline-capable scanner app, and Redis for running several servers.

61. What did you learn most? That correctness under concurrency and security must be designed and tested on the server, and that small, tested steps make a big project manageable.

62. What was the hardest part? The booking engine: holds, payments that arrive late, refunds, deadlocks and proving with 100 parallel threads that it never oversells.